

Enviar imágenes y PDFs desde el cliente

Cómo **enviar** un archivo (imagen o PDF) en un mensaje del cliente sobre el socket de MoP — inline en base64, sin subir a storage — y cómo **recibir** los que manda el operador.

✓ Verificado end-to-end • 2026-07-13

El socket ahora acepta adjuntos **embebidos en base64** dentro del mismo `client_event` que ya usás para mandar texto. Vos mandás los bytes del archivo; el backend de MoP lo sube a storage, genera la URL y lo deja visible para el operador. No necesitás pedir signed URLs ni subir a GCS.



NO ES CURL

El transporte es **Socket.IO (WebSocket)**, no HTTP plano — no se puede mandar el `emit` con `curl`. Los ejemplos son con `socket.io-client` (el mismo cliente que ya usás para el chat). Al final hay un script Node.js completo y ejecutable para probar.

1 Conexión

Conectate con tu `SOCKET_KEY` y esperá el evento `hello` antes de mandar nada.

```
import { io } from 'socket.io-client';

const socket = io('wss://socket.momentofpeople.com', {
  path: '/v1/socket/',
  transports: ['websocket'],
  auth: { key: SOCKET_KEY }, // tu key (la que ya usás)
});

socket.on('hello', () => {
  // listo para mandar mensajes
});
```

2 Mandar un mensaje con adjunto

Leé el archivo, pasalo a base64 (**sin** el prefijo `data:...;base64,`) y mandalo en `attachments[]`. El callback de `ack` te confirma que el server lo aceptó.

```
// `file` es un File (browser) o un Buffer (Node)
const base64 = /* browser */ await fileToBase64(file);
               /* node    */ buffer.toString('base64');

socket.emit('client_event', {
  schema: 'mop.client.message.inbound/v1',
  idempotencyKey: crypto.randomUUID(), // dedupe entre reconexiones
  payload: {
    externalUserId: 'br-usuario-123', // id estable de tu usuario
    user: { name: 'Juan Pérez', phone: '+5491133445566' },
    text: 'Te mando el comprobante', // requerido, mín. 1 caracter
    attachments: [{
      type: 'file', // 'image' | 'file'
      data: base64, // base64 crudo
      mime: 'application/pdf',
      filename: 'comprobante.pdf',
    }],
  },
}, (ack) => {
  if (ack?.ok) console.log('enviado', ack.eventId);
  else console.error('rechazado', ack);
});
```

RESPUESTA ESPERADA

`{ ok: true, eventId: "0b89391d-..." }` — el server aceptó y encoló el mensaje. Si algo está mal, `ok: false` con `error` y `details`.

3 Campos del `payload`

CAMPO		DETALLE
<code>externalUserId</code>	requerido	Id estable de tu usuario (string, 1–128).
<code>user.name</code>	requerido	Nombre a mostrar (1–200).
<code>user.phone</code>	phone o email	E.164, ej. <code>+5491133445566</code> . Hace falta phone o email: el CRM rechaza el contacto sin al menos uno.
<code>user.email</code>	alt.	Email válido — alternativa al phone.
<code>text</code>	requerido	Texto del mensaje, mín. 1 caracter (hasta 8000). Si solo mandás archivo, poné algo como <code>"[archivo adjunto]"</code> .
<code>attachments[]</code>	opcional	Hasta 10 adjuntos por mensaje.
• <code>type</code>	requerido	<code>'image'</code> o <code>'file'</code> (PDF).
• <code>data</code>	requerido*	Base64 crudo del archivo (sin <code>data:</code> URI). *o <code>url</code> si ya lo subiste.
• <code>mime</code>	recomendado	<code>application/pdf</code> , <code>image/jpeg</code> , <code>image/png</code> , <code>image/webp</code> .
• <code>filename</code>	recomendado	Nombre para mostrar / descarga.

LÍMITE DE TAMAÑO

El base64 (`data`) viaja dentro del evento por el bus de mensajes, que tope en 10 MB. Mantené los archivos por debajo de **~6 MB** (≈ 8M caracteres de base64). Para archivos más grandes, subilos por tu cuenta y mandá `url` en vez de `data`.

4 Script de prueba (Node.js)

Ejecutable tal cual — conecta, manda un PDF de ejemplo en base64 y te imprime el `ack`. Reemplazá `SOCKET_KEY`.

```
// test.mjs - node test.mjs
import { io } from 'socket.io-client';
import { readFileSync } from 'node:fs';
import { randomUUID } from 'node:crypto';

const SOCKET_KEY = '<tu-key>';
const base64 = readFileSync('./comprobante.pdf').toString('base64');

const socket = io('wss://socket.momentofpeople.com', {
  path: '/v1/socket/', transports: ['websocket'], auth: { key: SOCKET_KEY },
});

socket.on('hello', () => {
  socket.emit('client_event', {
    schema: 'mop.client.message.inbound/v1',
    idempotencyKey: randomUUID(),
    payload: {
      externalUserId: 'br-test-1',
      user: { name: 'Test', phone: '+5491133445566' },
      text: 'PDF de prueba',
      attachments: [{ type: 'file', data: base64, mime: 'application/pdf', fi
    },
  }, (ack) => { console.log(ack); socket.close(); });
});

socket.on('connect_error', (e) => console.error('connect_error', e.message));
```

ALTERNATIVA: ENVIAR POR URL

Si ya tenés el archivo subido, en vez de `data` mandá `url` (pública y accesible):

`{ type: 'image', url: 'https://.../foto.jpg', mime: 'image/jpeg' }`. Cada adjunto necesita `data` o `url` — al menos uno.

5 Recibir imágenes y documentos

Los mensajes del operador (y del bot) llegan por el evento `event`. Cuando el `schema` es `mop.message.outbound/v1`, lee `payload.text` y `payload.attachments`. Los adjuntos llegan como **URLs públicas** (no base64) — las renderizás directo.

```
socket.on('event', (e) => {
  if (e.schema !== 'mop.message.outbound/v1') return;

  const { text, attachments } = e.payload;
  if (text) render(text);

  for (const att of attachments ?? []) {
    if (att.type === 'image') show(``);
    else show(`<a href="${att.url}">${att.filename} ?? 'D
  }
});
```

Forma del adjunto recibido

```
{
  type: 'image' | 'file',
  url: 'https://storage.googleapis.com/.../archivo.pdf', // pública, direc
  mime: 'application/pdf', // opcional
}
```

Renderizá por `type`: `<img>` para `image`, un link o descarga para `file`. Un mensaje puede traer `text`, adjuntos, o ambos.

DEDUPE

En reconexiones el server puede reenviar el mismo evento. Deduplicá por `e.eventId` para no pintar el mensaje dos veces.

MoP · momentofpeople.com – dudas de integración: escribinos al equipo de plataforma.